AI Assignment

Build a Basic Document Q&A Bot

Using RAG (Retrieval-Augmented Generation)

Topic	RAG Pipeline — Basic Document Q&A Bot
Level	Beginner — AI Engineering Intern
Duration	3 days
Language	Python 3.11 or higher

1. Assignment Overview

You are required to build a working Retrieval-Augmented Generation (RAG) pipeline from scratch. The system must allow a user to ask natural language questions against a collection of documents and receive accurate, grounded answers — with clear references to the source material.

This assignment tests your ability to understand and implement the full RAG pipeline independently — from document ingestion and chunking, through embedding and vector storage, to retrieval and LLM-based answer generation. There is no starter code. You are expected to make your own architecture decisions and justify them.

2. Objective

By the end of this assignment, you must have built a functional Q&A bot that:

- Ingests 4 to 5 documents of your choice (PDF, TXT, or DOCX format)
- Chunks, embeds, and stores them in a vector database
- Accepts a user question as input and retrieves the most relevant document chunks
- Generates a grounded answer using a language model with source citations
- Runs end-to-end from the command line or a simple web UI

3. Documents to Use

You must select and include 4 to 5 documents as your knowledge base. These can be on any topic you choose (e.g. science, business, technology, law, or your own interest area). The documents must meet the following criteria:

- Each document must be at least 2 pages or 500 words in length
- At least one document must be in PDF format
- Documents must be meaningful and non-trivial (not placeholder or lorem ipsum text)
- All documents must be committed to your GitHub repository under a /data or /docs folder

You are encouraged to pick documents that make for interesting and diverse queries. Avoid using single-topic documents that are too narrow to generate meaningful questions.

4. Technical Requirements

Your implementation must cover all of the following components. You are free to choose the specific libraries or tools, but each component must be present and functional.

4.1 Document Ingestion

- Load documents from a local folder or file paths
- Support at least PDF format; support for DOCX or TXT is a bonus
- Extract clean text, stripping headers, footers, and page numbers where possible

4.2 Text Chunking

- Split documents into chunks before embedding
- You must choose a chunking strategy (fixed-size, sentence-based, or paragraph-based) and document your choice in the README
- Chunks must have appropriate overlap to avoid losing context at boundaries
- Store chunk metadata: source filename and page number or section

4.3 Embedding

- Embed each chunk using an embedding model of your choice
- All embedding calls must be batched — do not embed one chunk at a time in a loop

4.4 Vector Database

- Store all embeddings in a vector database — FAISS, ChromaDB, Qdrant, or Weaviate are all acceptable
- The vector store must persist to disk so it does not re-index on every run
- Implement a clear separation between the indexing step and the querying step

4.5 Retrieval

- On receiving a user query, embed the query and perform a similarity search
- Retrieve the top-k most relevant chunks (k should be configurable)
- Pass the retrieved chunks as context to the language model

4.6 Answer Generation

- Use an LLM to generate a final answer from the retrieved context
- Acceptable LLMs: OpenAI GPT-3.5 or GPT-4, Google Gemini, Anthropic Claude, or any open-source model via Ollama
- The answer must include a source citation — at minimum the filename and relevant section or page number
- Do not allow the model to answer from its own training data if the answer is not in the retrieved context

4.7 User Interface

- At minimum, the bot must run from the command line as an interactive loop
- A simple Streamlit or Gradio web UI is a bonus — not required
- The interface must display the answer and the source chunks used

5. Deliverables

You must submit all four of the following. Submissions missing any item will be considered incomplete.

#	Deliverable	Requirements	Format
1	GitHub Repository	Public repo with all source code, documents, and README. Must include a clean commit history — no single giant commit.	GitHub URL
2	README File	Detailed instructions to install dependencies and run the project. Must document architecture decisions (chunking strategy, embedding model, vector DB choice).	README.md in repo

3	Document Collection	4 to 5 documents committed to the /data or /docs folder. At least one must be a PDF. Must be non-trivial content.	Files in repo
4	Screen Recording	Loom video or screen recording (3 to 8 minutes) demonstrating: app startup, at least 5 different queries with output and citations shown clearly.	Loom or MP4 link

6. README File Requirements

Your README.md must be professional and complete. A recruiter or engineer should be able to clone your repo and run the project without any additional guidance from you. The README must include all of the following sections:

1. Project title and a 2 to 3 sentence description of what the bot does
2. Tech stack — list every library and tool used with version numbers
3. Architecture overview — describe or diagram the RAG pipeline (ingestion, chunking, embedding, retrieval, generation)
4. Chunking strategy — which strategy you chose and why
5. Embedding model and vector database — which you chose and why
6. Setup instructions — step by step from git clone to running the bot
7. Environment variables — list all required API keys and how to set them (never commit actual keys)
8. Example queries — at least 5 sample questions and expected answer themes
9. Known limitations — what the bot struggles with and why

7. Screen Recording Requirements

Record a Loom video or screen recording (between 3 and 8 minutes) that covers all of the following. Recordings that skip any step will be penalised in scoring.

- Show your project folder structure and briefly explain each component
- Run the indexing step — show the documents being loaded, chunked, and stored into the vector database
- Run the querying step — show the bot starting up and accepting input
- Ask at least 5 different questions that span across at least 2 different documents
- For each answer, show the retrieved source chunks and citations clearly
- Ask one question that is NOT answerable from your documents — show how the bot handles it
- Briefly explain one technical decision you made (e.g. why you chose that chunking strategy or vector DB)

8. Rules and Integrity

Permitted

- Using any open-source libraries or frameworks (LangChain, LlamaIndex, Haystack, etc.)
- Referencing official documentation, tutorials, and blog posts
- Using AI coding assistants (ChatGPT, GitHub Copilot, Claude, etc.) for syntax help

Not Permitted

- Using an API key in your committed code (use .env or environment variables)
- Submitting a recording that does not show live output — pre-recorded demos with fake output will be disqualified

Originality is valued. A simpler implementation that you fully understand is worth more than a complex one you cannot explain.

9. How to Submit

Send the following to your hiring contact or via the submission form provided:

10. GitHub repository URL (must be public and accessible)
11. Loom or screen recording URL (must be accessible without login)

Deadline: As communicated by the hiring team. Late submissions without prior notice will not be reviewed.

Good luck. Build something you are proud to explain.
We look forward to reviewing your work.